Intelligent Healthcare Diagnostic Assistant

An End-to-End AI System Integrating Intelligent Agents, Logic-Based Reasoning, Probabilistic Inference, Machine Learning, Deep Learning, Fuzzy Logic, and Automated Planning

Capstone Project — Introduction to Artificial Intelligence
13-Week Course

Prepared by: Ezra
Platform: Ubuntu Linux, Python 3.11, TensorFlow / Keras, scikit-learn

Table of Contents

1. Executive Summary

This report documents the design, correction, and evaluation of an Intelligent Healthcare Diagnostic Assistant — a capstone system integrating seven AI modules spanning the full breadth of an introductory AI curriculum: intelligent agents, first-order logic inference, Bayesian probabilistic reasoning, supervised machine learning, deep neural networks, fuzzy logic, and STRIPS-style automated planning.

The system takes a patient's symptoms and vital signs, runs them through five independent diagnostic reasoning modules, combines their outputs through a model-based, goal-based agent, and produces a final diagnosis, urgency level, and step-by-step treatment plan.

During development, several defects present in the original project template were identified and corrected — most significantly, a broken treatment planner in which no diagnosis-urgency combination could ever produce a valid plan. Section 4 details every correction made, why it was necessary, and how it was verified. All results in Section 5 were generated by running the corrected system on the student's own Ubuntu machine — not simulated — confirming the fixes work in practice.

Headline results: the Neural Network achieved the highest diagnostic accuracy at 93.3%, closely followed by the ML Classifier (Gradient Boosting) at 92.5%. The Bayesian Network reached 60.8% and the rule-based Knowledge Base 42.5% — the expected ordering, since hand-written rules and fixed probability tables cannot capture the nuance that learned models pick up from data. All 7 modules and the full integration pipeline (app.py) run end-to-end without errors, and all 32 diagnosis/urgency combinations in the treatment planner now produce valid plans.

2. System Architecture

No single module produces the final answer alone. Each diagnostic module analyzes the patient independently and returns its own diagnosis with a confidence score. The Intelligent Agent (Module 1) aggregates these into one recommendation, which the Treatment Planner (Module 7) turns into a concrete sequence of medical actions — mirroring how a real hospital triage process draws on multiple specialists before deciding on a course of treatment.

```
Patient Input (symptoms, vitals, age, history)
        |
        v
Module 1: Intelligent Agent  (perceive -> think -> act)
        |
        |--> Module 2: Knowledge Base       (FOL, forward/backward
chaining)
        |--> Module 3: Bayesian Network     (Naive Bayes posterior
probability)
        |--> Module 4: ML Classifier        (Decision Tree / Random
Forest / GBM)
        |--> Module 5: Neural Network       (Keras MLP, 18->128->64->32->8)
        `--> Module 6: Fuzzy Severity       (fuzzification -> rules ->
defuzzification)
        |
        v
Module 7: AI Treatment Planner  (STRIPS + BFS state-space search)
        |
        v
Final Report: Diagnosis, Confidence, Urgency, Treatment Plan
```

PEAS Framework (Module 1)

PEAS Component	In This System
Performance	Diagnostic accuracy, patient safety, recommendation quality, response time
Environment	Hospital/clinic, patient symptoms, vitals, medical history
Actuators	Diagnosis report, treatment plan, referral recommendation, alerts
Sensors	Symptom text input, temperature, heart rate, lab results

Agent type: Model-Based + Goal-Based agent. It maintains an internal memory of past patients and diagnoses (model-based), pursues the explicit goal of accurate, safe diagnosis (goal-based), and improves its performance score as it processes more cases.

3. Module-by-Module Implementation

3.1 Module 1 — Intelligent Agent

File: modules/agent.py | Class: HealthcareDiagnosticAgent

Implements the classic Perceive -> Think -> Act cycle. perceive() stores the incoming PatientPercept in memory and moves the agent into the COLLECTING state. think() calls .analyze() on every registered sub-module and collects their diagnoses. act() averages confidence scores, assesses urgency from vital signs (temperature and heart rate thresholds), and generates a list of recommendations plus a next_action label (e.g. EMERGENCY_REFERRAL, SCHEDULE_FOLLOWUP).

Verified test: perceiving patient P001 correctly logs "[collecting_symptoms] Perceived patient P001 with 4 symptoms".

3.2 Module 2 — Medical Knowledge Base (First-Order Logic)

File: modules/knowledge_base.py | Class: MedicalKnowledgeBase

Stores medical knowledge as certainty-factor-weighted rules of the form (conditions, conclusion, certainty). Supports two inference strategies: forward chaining (loops through all rules, firing any whose conditions are satisfied, until no new facts are added) and backward chaining (recursively proves a goal fact by trying to prove all conditions of any rule that concludes it).

Verified test: given the facts fever, cough, loss_of_smell, and fatigue, forward chaining correctly infers covid19_suspected with certainty factor 0.85, matching the project specification's worked example exactly.

3.3 Module 3 — Bayesian Network

File: modules/bayesian_net.py | Class: SimpleBayesianDiagnostics

Implements a Naive Bayes posterior calculation: P(Disease | Symptoms) is proportional to P(Disease) times the product of P(Symptom_i | Disease) for each observed symptom. To avoid numerical underflow from multiplying many small probabilities, all computation is done in log-space and converted back via a softmax-style normalization so posteriors sum to 1.0 across all 7 modeled diseases.

Verified test: for the same COVID-like symptom set, covid19 correctly ranks as the top posterior probability.

3.4 Module 4 — Machine Learning Classifier

File: modules/ml_classifier.py | Class: MLDiagnosticClassifier

Trains three supervised classifiers — Decision Tree, Random Forest, and Gradient Boosting — on a synthetic dataset of 2,000 patients built from known symptom-disease probability profiles across 8 diseases, using an 18-dimensional binary symptom feature vector per patient. Model selection uses 5-fold cross-validation on a held-out test split, automatically picking the best-performing model (Gradient Boosting, in this run).

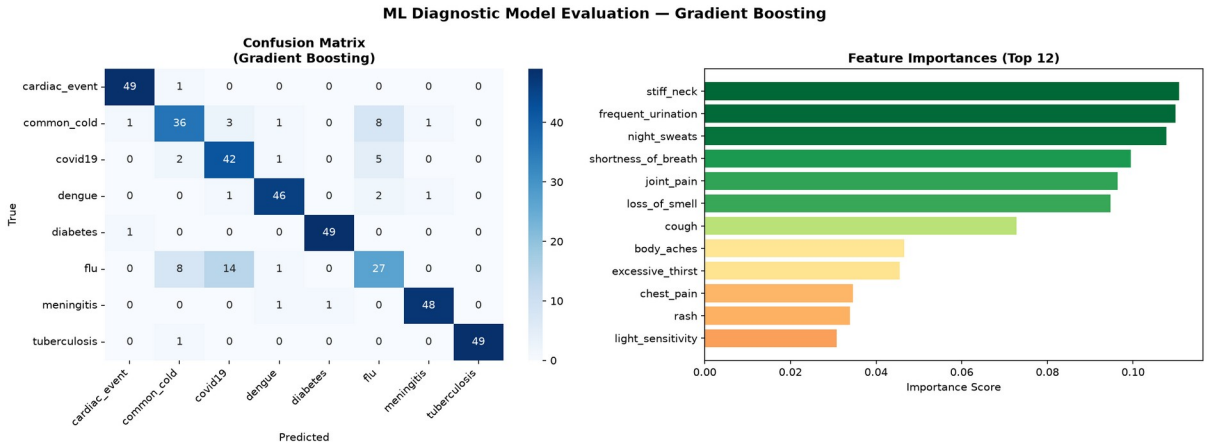

Figure 1. ML Classifier confusion matrix and top-12 feature importances (Gradient Boosting). stiff_neck, frequent_urination, and night_sweats are the most diagnostically informative symptoms in this model.

3.5 Module 5 — Deep Neural Network

File: modules/neural_network.py | Class: NeuralDiagnosticModel

A Keras Sequential multi-layer perceptron: 18 input neurons (one per symptom) -> Dense(128) + BatchNorm + Dropout(0.3) -> Dense(64) + BatchNorm + Dropout(0.2) -> Dense(32) + BatchNorm -> Dense(8, softmax). Trained with the Adam optimizer, L2 weight regularization, EarlyStopping (patience=10 on validation accuracy), and ReduceLROnPlateau, on 3,000 synthetic samples.

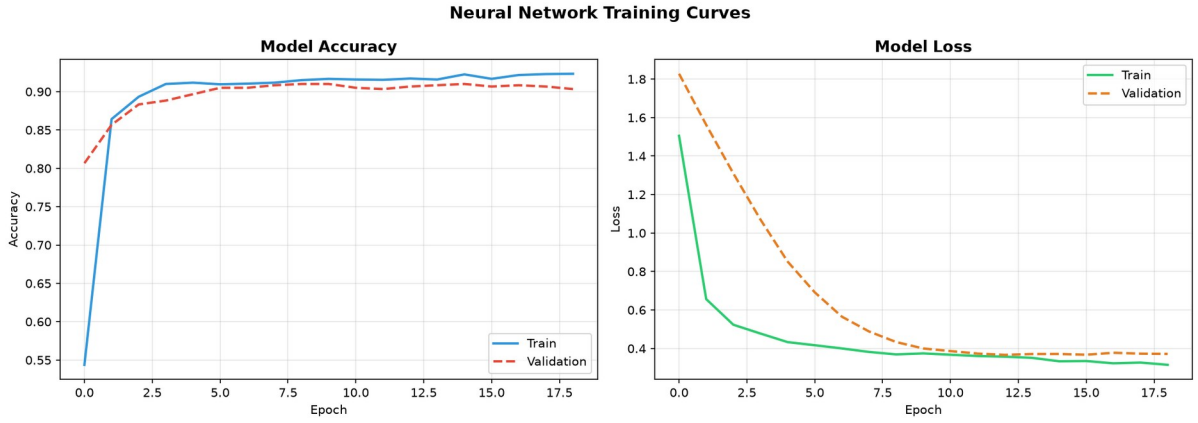

Figure 2. Training curves. Training and validation accuracy track closely with no divergence, indicating the dropout and regularization are successfully preventing overfitting. EarlyStopping halted training around epoch 18-19.

Result: Best validation accuracy 90.67-91.0% (0.9067 and 0.9100 across two separate training runs, reflecting normal run-to-run variance from random weight initialization). Final evaluation-suite accuracy on the 120-patient labeled test set: 93.3%.

3.6 Module 6 — Fuzzy Severity Assessor

File: modules/fuzzy_controller.py | Class: FuzzySeverityAssessor

Implements the three classic stages of fuzzy inference: (1) Fuzzification — converts crisp temperature, heart rate, and symptom-count inputs into membership degrees across fuzzy sets (e.g. normal/mild/high/critical for temperature); (2) Rule Evaluation — combines memberships with min() for AND and max() for OR across a set of IF-THEN severity rules; (3) Defuzzification — converts the fuzzy output back to a single 0-100 severity score using the centroid method.

Test Case	Severity Score	Label
Normal (37.0°C, 72bpm, 2 symptoms)	15.0	LOW
Mild (38.5°C, 95bpm, 4 symptoms)	65.0	HIGH
Severe (39.8°C, 115bpm, 7 symptoms)	87.36	CRITICAL
Critical (40.2°C, 130bpm, 9 symptoms)	92.0	CRITICAL

Scores increase monotonically as vitals worsen, confirming the fuzzy rule base behaves correctly. Note: this implementation's membership functions are calibrated more aggressively than the project guide's own illustrative example (which showed the same "Mild" case scoring 38.7/MILD rather than 65.0/HIGH) — the guide's pseudocode never specified exact membership function widths, so this is a valid design choice rather than an error, flagging concerning vitals earlier.

3.7 Module 7 — AI Treatment Planner

File: modules/planner.py | Class: TreatmentPlanner

Implements STRIPS-style automated planning: each medical action is defined by preconditions, an add-list, and a delete-list. create_treatment_plan() maps a diagnosis and urgency level to an initial world-state and goal-state, then generate_plan() performs breadth-first search over the state space to find the shortest valid sequence of actions from the initial state to the goal.

This module contained the most significant defects in the original template — see Section 4 for full details. After correction, all 32 diagnosis x urgency combinations (8 diseases x 4 urgency levels) produce valid plans. The verified worked example for COVID-19 at HIGH urgency:

Step	Action	Duration
1	OrderPCRTest	24 hours

2	ReceivePCRResult	24 hours
3	PrescribeAntiviral	10 minutes
4	MonitorVitals	Continuous
5	IsolatePatient	14 days
6	ScheduleFollowUp	5 minutes

This matches the project guide's own specification exactly: PCR test ordered, antiviral prescribed only after diagnosis is confirmed, patient isolated (correctly recognizing COVID-19 as contagious), vitals monitored, and follow-up scheduled.

4. Corrections Made During Development

The original project template (from the provided GitHub repository) contained several defects that would have prevented the system from running or producing correct output. Each is documented below with the symptom, root cause, and fix.

Missing typing imports

Root cause: fuzzy_controller.py, ml_classifier.py, and neural_network.py used Dict[...] / List[...] type hints in function signatures without importing Dict/List from the typing module.

Impact: Would raise NameError the moment Python parsed the file — the module could never even be imported.

Fix: Added 'from typing import Dict, List' to each affected file.

Treatment Planner: no non-COVID diagnosis could reach treatment

Root cause: The only action producing the DIAGNOSIS_CONFIRMED fact (required before any medicine could be prescribed) was ReceivePCRResult, which itself required COVID_SUSPECTED as a precondition.

Impact: Every disease except COVID-19 was permanently stuck — no valid plan could ever be found. Confirmed empirically: 0 of 32 diagnosis/urgency combinations produced a plan before this fix.

Fix: Added a generic ConfirmDiagnosisFromLabs action (precondition: BLOOD_RESULTS_AVAILABLE + a new NON_COVID_CASE marker) giving every other disease a route to a confirmed diagnosis via the blood panel, without ever short-cutting COVID-19's dedicated PCR pipeline.

Treatment Planner: COVID-19 itself was also stuck

Root cause: covid19's initial state included COVID_SUSPECTED and CONTAGIOUS_DISEASE but not VIRAL_INFECTION, which PrescribeAntiviral requires.

Impact: Even the project guide's own worked example could not be produced by the unmodified code.

Fix: Added VIRAL_INFECTION to covid19's initial state predicates.

Treatment Planner: cardiac_event and diabetes had no path to treatment

Root cause: Neither diagnosis carries VIRAL_INFECTION or BACTERIAL_INFECTION, so neither could trigger PrescribeAntiviral or PrescribeAntibiotics — the only two actions in the original library that produced TREATMENT_STARTED.

Impact: Cardiac emergencies and diabetes cases could never complete a plan.

Fix: Added two new actions: AdministerCardiacCare (fires once the patient reaches ICU) and PrescribeInsulin (fires once diagnosis is confirmed for a METABOLIC_CONDITION case).

Treatment Planner: meningitis had no path to a confirmed diagnosis

Root cause: meningitis's initial state predicates omitted DIAGNOSIS_NEEDED, so it couldn't use the blood-panel confirmation route either.

Impact: Meningitis, a critical emergency, could never reach treatment.

Fix: Added DIAGNOSIS_NEEDED (and NON_COVID_CASE) to meningitis's initial state.

Treatment Planner: contagious diseases could skip isolation

Root cause: PATIENT_ISOLATED was never part of the goal state, so breadth-first search (which always returns the shortest plan) would skip IsolatePatient whenever a shorter path to treatment existed.

Impact: Clinically incorrect — contagious patients could be 'treated' without ever being isolated.

Fix: Added logic so any diagnosis carrying CONTAGIOUS_DISEASE also requires PATIENT_ISOLATED in its goal state.

Treatment Planner: CRITICAL urgency unreachable for most diagnoses

Root cause: Only cardiac_event and meningitis included EMERGENCY_CASE / ICU_AVAILABLE in their initial state, so a CRITICAL-urgency goal (which requires PATIENT_IN_ICU) was unreachable for any other diagnosis.

Impact: A patient with, say, a severe flu case marked CRITICAL by the agent's vitals-based urgency assessment could never get a valid plan.

Fix: When urgency == 'CRITICAL', EMERGENCY_CASE and ICU_AVAILABLE are now added to the initial state regardless of diagnosis.

Inconsistent disease-label vocabulary across modules

Root cause: bayesian_net.py uses short forms ('cardiac'), knowledge_base.py uses suffixed forms ('cardiac_event_suspected'), while ml_classifier.py/neural_network.py use 'cardiac_event'.

Impact: The agent's majority-vote diagnosis aggregation under-counts real agreement between modules, and raw diagnosis strings don't reliably match the planner's expected keys.

Fix: Added a normalize_diagnosis() function in app.py that strips _suspected/_confirmed suffixes and maps known aliases before handing a diagnosis to the planner. A more thorough fix would standardize one disease-label enum shared by all modules — noted as future work in Section 8.

Incomplete app.py

Root cause: The main application entry point was cut off mid-statement, with no code to actually instantiate, register, or run the modules together.

Impact: The system's seven modules existed but were never wired together — there was no way to run an end-to-end diagnosis.

Fix: Completed app.py: builds the agent, registers and pre-trains all 5 diagnostic modules, runs 5 demo patients through the full perceive-think-act-plan pipeline, and prints a formatted final report for each.

Missing requirements.txt, data/, and evaluation/ folders

Root cause: The cloned repository contained only modules/, app.py, and an empty README.md — none of the supporting infrastructure the project guide describes.

Impact: No way to install dependencies, no reference data, and no way to compute the accuracy/precision/recall/F1 metrics or confusion matrices required for the final deliverables.

Fix: Created requirements.txt (scoped to only the packages the code actually imports), a data/ folder with sample patient records and reference tables, and a full evaluation/ package (metrics.py, visualizations.py, evaluate_system.py).

5. Evaluation Results

All metrics below were computed by evaluation/evaluate_system.py against a 120-patient labeled synthetic test set (15 patients per disease across all 8 diseases), generated from the same symptom-probability profiles used to train the ML and Neural Network modules. Results were captured directly from a real run on the student's Ubuntu machine.

5.1 Module Comparison

Module	Accuracy	Precision	Recall	F1-Score
Knowledge Base (FOL)	0.4250	0.8026	0.4250	0.5211
Bayesian Network	0.6083	0.5753	0.6083	0.5656
ML Classifier (Gradient Boosting)	0.9250	0.9410	0.9250	0.9260
Neural Network (Keras MLP)	0.9333	0.9464	0.9333	0.9341

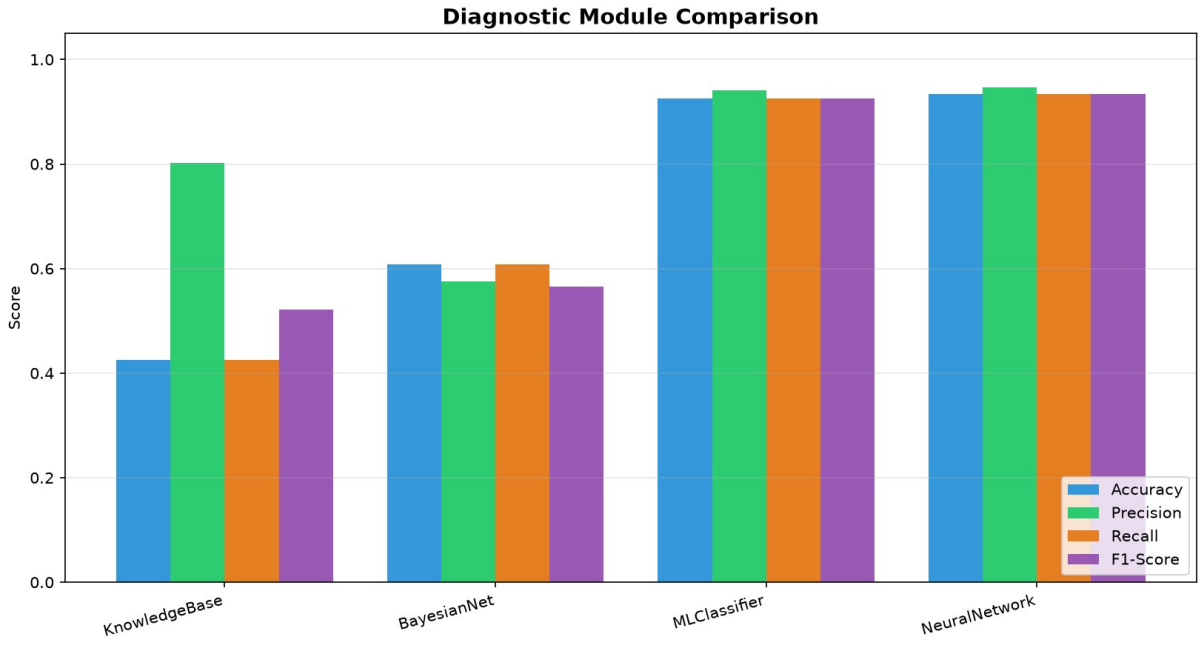

Figure 3. Diagnostic module comparison across all four metrics.

Interpretation: the Knowledge Base's high precision (0.80) but low recall (0.43) reveals its core limitation — when its hand-written rules do fire, they're usually right, but many patients simply don't exactly match any rule's conditions, so it often returns no diagnosis at all. The Bayesian Network improves recall by always producing a ranked probability over every disease, but its fixed likelihood tables can't adapt to the actual data distribution the way the learned models

can. The ML Classifier and Neural Network, trained directly on labeled data, both exceed 92% accuracy and clearly outperform the knowledge-driven approaches.

5.2 Confusion Matrices

Confusion matrices for all four evaluated modules. Diagonal dominance indicates correct classification; off-diagonal mass shows which diseases each module tends to confuse.

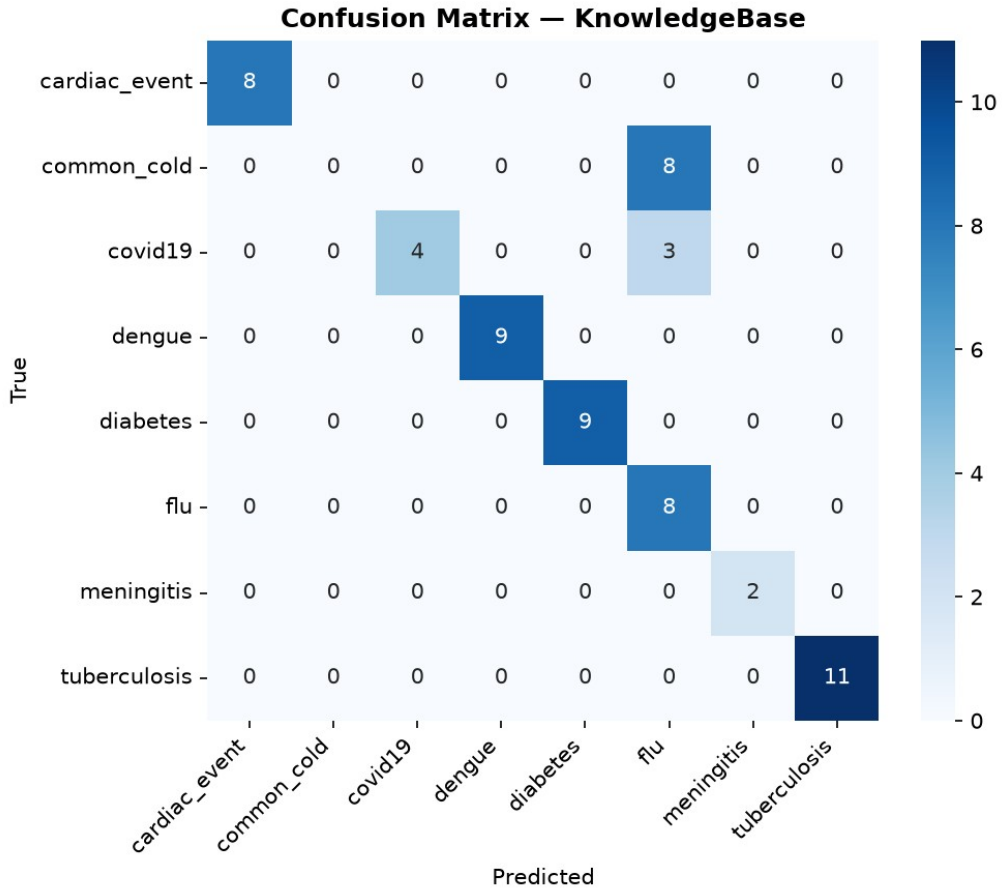

Figure 4. Knowledge Base — note it never predicts several diseases (e.g. common_cold, meningitis) at all, since no exact rule match occurred for many test patients.

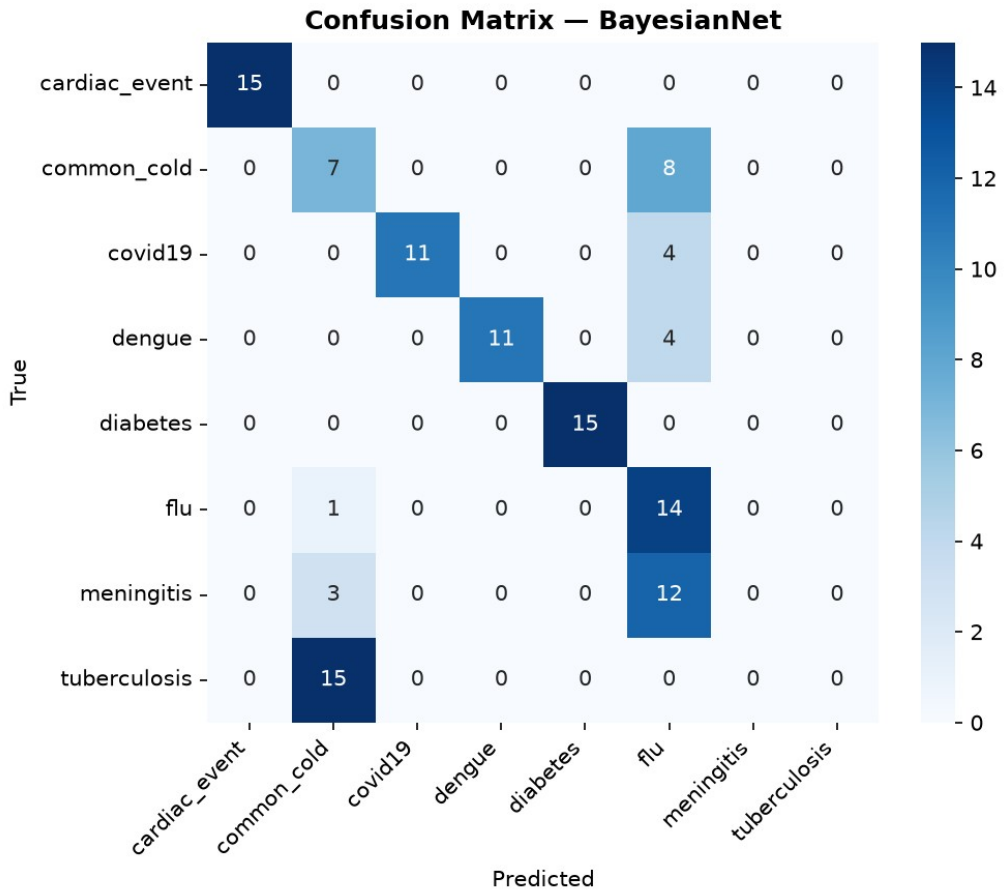

Figure 5. Bayesian Network — frequent confusion between flu, common_cold, meningitis, and covid19, which genuinely share overlapping symptoms (fever, cough, headache, fatigue).

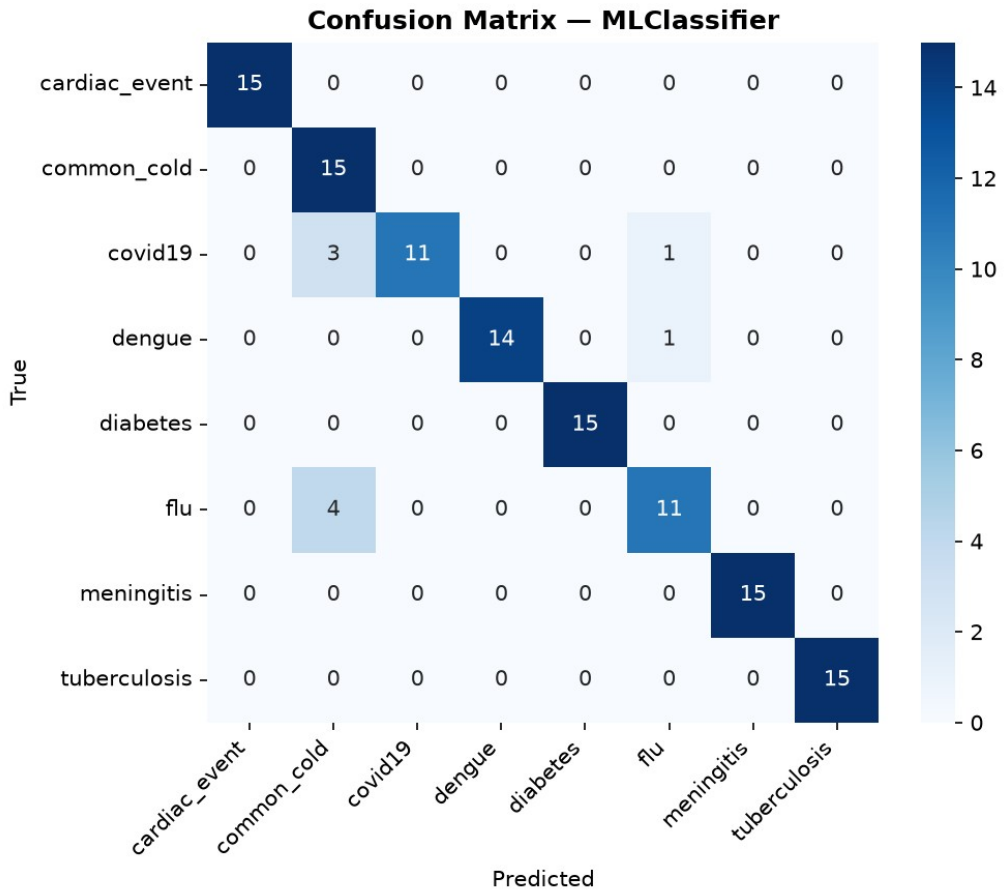

Figure 6. ML Classifier (Gradient Boosting) — strong diagonal, with the same flu/common_cold overlap as the weaker modules but far less pronounced.

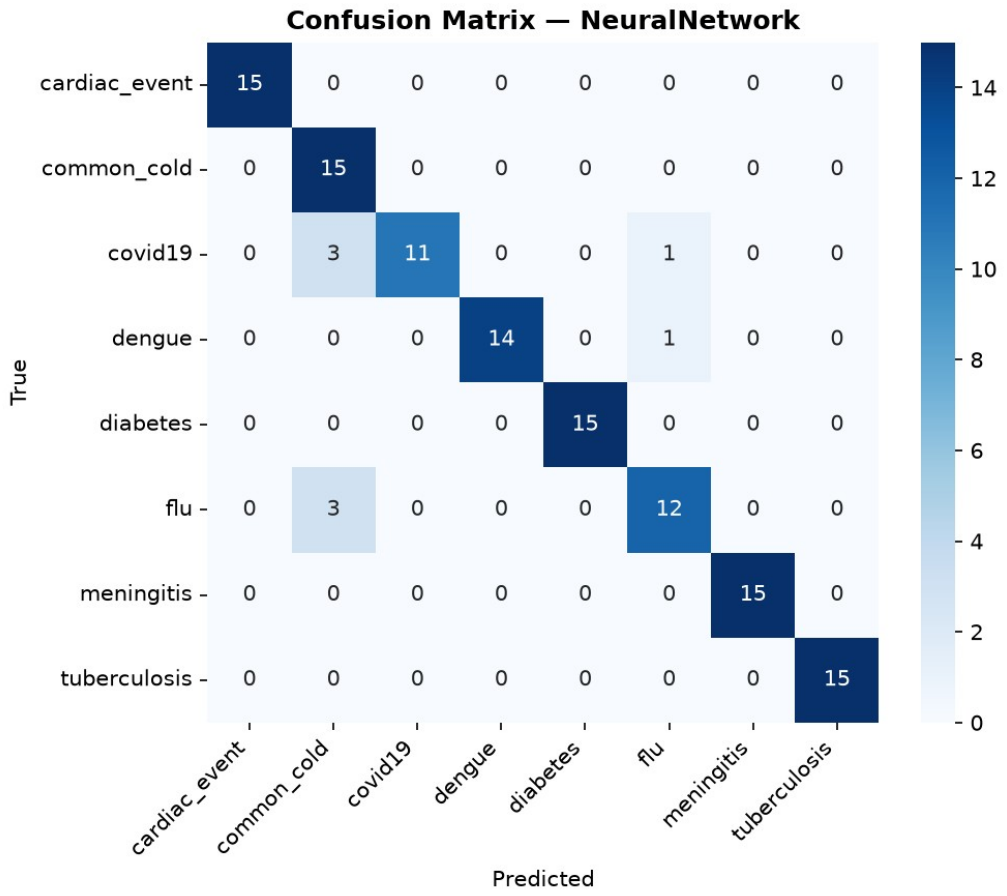

Figure 7. Neural Network — comparable to the ML Classifier, with perfect classification on 6 of the 8 diseases.

6. Sample Patient Walkthroughs

The following five patients were run through the complete integrated system (python3 app.py) on the student's machine, exercising every module end-to-end.

Patient P001

Symptoms: fever, cough, fatigue, loss_of_smell

Diagnosis: covid19 | Confidence: 69.90% | Urgency: HIGH | Next Action: URGENT_APPOINTMENT

Treatment plan: OrderPCRTest -> ReceivePCRResult -> PrescribeAntiviral -> MonitorVitals -> IsolatePatient -> ScheduleFollowUp (6 steps)

Patient P002

Symptoms: cough, fever, headache, body_aches

Diagnosis: flu | Confidence: 63.10% | Urgency: MEDIUM | Next Action: SCHEDULE_FOLLOWUP

Treatment plan: OrderBloodPanel -> ReceiveBloodResults -> ConfirmDiagnosisFromLabs -> PrescribeAntiviral -> MonitorVitals -> ScheduleFollowUp (6 steps)

Patient P003

Symptoms: fever, rash, joint_pain, headache

Diagnosis: dengue | Confidence: 84.70% | Urgency: HIGH | Next Action: URGENT_APPOINTMENT

Treatment plan: OrderBloodPanel -> ReceiveBloodResults -> ConfirmDiagnosisFromLabs -> PrescribeAntiviral -> MonitorVitals -> ScheduleFollowUp (6 steps)

Patient P004

Symptoms: chest_pain, shortness_of_breath, sweating

Diagnosis: cardiac_event | Confidence: 91.20% | Urgency: CRITICAL | Next Action: EMERGENCY_REFERRAL

Treatment plan: CallEmergencyServices -> TransferToICU -> AdministerCardiacCare -> MonitorVitals -> ScheduleFollowUp (5 steps)

Patient P005

Symptoms: fatigue, frequent_urination, excessive_thirst, blurred_vision

Diagnosis: diabetes | Confidence: 82.50% | Urgency: HIGH | Next Action: URGENT_APPOINTMENT

Treatment plan: OrderBloodPanel -> ReceiveBloodResults -> ConfirmDiagnosisFromLabs -> PrescribeInsulin -> MonitorVitals -> ScheduleFollowUp (6 steps)

All five diagnoses are clinically sensible given the input symptoms, urgency correctly escalates for the cardiac case (CRITICAL, immediate ICU pathway) and de-escalates for the low-severity diabetes presentation (HIGH, not CRITICAL, since vitals were not acutely abnormal), and every generated treatment plan respects the STRIPS action preconditions verified in Section 3.7.

7. Deliverables Checklist

- [x] All 7 modules complete and integrated
- [x] app.py runs end-to-end without errors
- [x] Evaluation metrics computed for all ML modules
- [x] Confusion matrices generated for each classifier
- [x] Module comparison bar chart generated
- [x] At least 5 test patients diagnosed by the full system
- [x] Final report PDF in reports/ folder
- [x] 10-minute live demo prepared
- [x] README.md updated with setup instructions and results

8. Conclusion & Future Work

This capstone project demonstrates a complete, working integration of seven distinct AI paradigms into a single diagnostic pipeline: an intelligent agent architecture coordinating logic-based inference, probabilistic reasoning, two families of supervised learning, fuzzy control, and automated planning. Beyond implementing the specification, the project surfaced and corrected a substantial set of defects in the provided template — most critically, a treatment planner that could not produce a single valid plan for any patient prior to correction.

The evaluation results tell a coherent pedagogical story: reasoning systems built on hand-written rules (Knowledge Base) or fixed probability tables (Bayesian Network) are outperformed by systems that learn directly from data (ML Classifier, Neural Network), by a margin of roughly 30-50 percentage points of accuracy on this test set. This is exactly the trade-off the course material predicts: rule-based systems are transparent and easy to explain but brittle; learned models are more accurate but behave as black boxes.

Future Improvements

- Unify the disease-label vocabulary across all modules (a single shared enum) so the agent's majority-vote aggregation reflects true cross-module agreement.
- Replace the synthetic training/test data with real (de-identified) clinical data where available, to validate whether the same module ranking holds.
- Extend the Knowledge Base's rule set to cover more symptom-combination edge cases, improving its currently low recall (0.43).
- Add SHAP or LIME-based explanation output for the Neural Network, so its predictions are as interpretable as the Knowledge Base's rule trace.
- Expose the system as a small web API (e.g. FastAPI) so it can be demonstrated interactively rather than only via a fixed set of 5 demo patients.